Deliverable 1 — Experimental Execution Summary

Replication of LoMix (NeurIPS 2025) on Synapse Multi-organ

Md. Irtiza Hossain
M.S. Computer Science & Engineering, BRAC University
mohammad.irtiza.hossain@gmail.com

6 September 2026

Paper	LoMix: Learnable Weighted Multi-Scale Logits Mixing for Medical Image Segmentation. Rahman & Marculescu, NeurIPS 2025.
Code	`github.com/SLDGroup/LoMix` @ commit `464a945`
Data	Synapse Multi-organ (BTCV), 9 classes, 224×224, authors' archive
Harness	`https://github.com/irtiza1999/LoMix-Reproducibility-Study`

The harness repository contains every run log and manifest cited below, the patched LoMix source alongside its unmodified `.orig` backups, the environment setup, and the analysis tools. Datasets, pre-trained weights and checkpoints are excluded by design; `setup/02_data_and_weights.md` reproduces them locally.

> Every number below came out of a run on this machine, and the log that produced it is in `logs/`. Nothing is copied from the paper into a results table. Where a run did not produce a number, the table says so.

1 Environment

Item	Value
GPU	NVIDIA GeForce GTX 960, 4 GB, capability 5.2 (Maxwell, no tensor cores)
Driver / CUDA	582.28 / CUDA 13.0 runtime; torch built against 12.1
OS	Windows 11 Pro 26200
Python / PyTorch	3.10.21 / 2.2.2+cu121
timm	0.6.12
Env recipe	`setup/01_create_env.ps1`, verified by `setup/verify_env.py`

Two deliberate departures from the repository README:

- **torch 2.2.2+cu121 rather than 1.11.0+cu113.** The wheel ships `sm_50` cubins, which run on this card's `sm_52`; verified by `torch.cuda.get_arch_list()` plus a live fp32/fp16 matmul.
- **A dedicated conda environment.** The machine's base environment carries timm 1.0.26, where `timm.models.layers.trunc_normal_tf_` and `timm.models.helpers.named_apply` have moved. The repository imports both.

An ordering trap worth recording: installing the repository's dependencies *after* the pinned torch lets pip re-resolve the unpinned `torch` requirement of `timm`/`thop`/`ptflops`/`torchprofile` and silently replace 2.2.2+cu121 with a CPU build, leaving the cu121 CUDA DLLs in place. The result is two torch dist-infos and `OSError: [WinError 127] ...c10_cuda.dll` at import. The setup script now installs torch last and pins the torch-dependent tools with `--no-deps`.

2 Deviations from the paper's protocol

Deviation	Paper	Here	Effect
Per-step batch	6	2	None on the loss; accumulation restores the effective batch
Grad. accumulation	none	3 (eff. batch 6)	Optimiser sees the paper's batch; BatchNorm statistics come from 2 samples, not 6
Precision	fp32	fp32	None (AMP measured 2.8–3.2× *slower* here, §4)
Epochs	300*	20	**Under-trained — the main caveat on every Dice below**
Validation	every epoch	every 2 epochs	Coarser best-checkpoint selection
Dataloader workers	8	2	Throughput only
Preprocessing	—	authors' mirror	The 14→9 label remap is correct as shipped and was left untouched; labels verified to span exactly 0–8 over 200 slices

*The paper's main text does not state an epoch budget; 300 is the default in `train_synapse_lomix.py`. It is cited here as the repository's schedule, not as a published figure.

Everything here is a **reduced-budget reproduction at 20 of the repository's default 300 epochs**. No Dice below should be compared with a published number without repeating that sentence. The paper's own results are averaged over at least three runs; mine are a single seed.

Patch applied by `tools/apply_low_vram_patch.py`, which rebuilds from `trainer.py.orig` every time and is reversible with `--revert`. The LoMix loss module, the learnable weights, the operation set and the network are untouched.

3 Stage A — efficiency verification

224×224, batch 1, 50 timed iterations after 10 warm-up. Source: `logs/bench_efficiency_Ishad.json`.

Encoder	Params (M)	Dec. (M)	GMACs	Latency (ms)	img/s	Peak VRAM
pvt_v2_b0	3.921	0.507	0.657	17.70 ± 2.75	56.5	34.1 MiB
pvt_v2_b2	26.773	1.914	4.324	36.77 ± 7.08	27.2	147.0 MiB

The decoder accounts for 1.914 M of 26.773 M parameters with `pvt_v2_b2`, consistent with EMCAD's published decoder cost — a useful check that the model is built as intended.

The zero-inference-overhead claim holds. The mixing weights are parameters of the loss module, not of `EMCADNet`, so all four supervision arms share a byte-identical inference graph. One measurement settles it for all of them.

The paper states this qualitatively rather than numerically — "the added parameters are negligible and used only during training, leaving the FLOPs, latency, and memory footprint at test-time unchanged" (§1) — and gives no inference-cost table in the main text, so there is no published figure to compare against. The measurement above is therefore a direct confirmation of the claim rather than a reproduction of a reported number: parameter count, GMACs and peak inference VRAM are identical across all four supervision settings. Absolute latency is not

comparable in any case, since the paper's experiments use an RTX A6000 (48 GB) and these use a GTX 960.

4 Stage B — what fits in 4 GB

Real forward + LoMix loss + backward + AdamW step on synthetic data, each trial in its own subprocess. Source: `logs/memfit_Ishad_lomix_sweep.json`. The sweep was run twice; **peak memory was identical to the byte, step time was not.**

Encoder	Batch	AMP	Peak VRAM	Step (r1)	Step (r2)	Note
pvt_v2_b0	2	off	1095.9	1034	936	
pvt_v2_b0	4	off	2130.2	1787	1709	
pvt_v2_b0	6	off	3180.1	6558	5870	over TDR watchdog
pvt_v2_b0	2	on	1047.9	3288	3321	AMP $\sim 3.2\times$ slower
pvt_v2_b2	1	off	978.5	640	779	
pvt_v2_b2	**2**	**off**	**1632.9**	**1046**	**1415**	**chosen**
pvt_v2_b2	3	off	2289.6	1432	2193	r2 crosses the watchdog
pvt_v2_b2	4	off	2941.4	3472	5398	over TDR watchdog
pvt_v2_b2	6	off	4268.3	7967	11144	>4096 MiB, WDDM paging
pvt_v2_b2	2	on	1513.8	2940	3969	AMP $\sim 2.8\times$ slower
pvt_v2_b2	4	on	—	—	—	`CUDNN_STATUS_EXECUTION_FAILED`

VRAM in MiB, step time in ms.

Chosen: `pvt_v2_b2`, batch 2, accumulation 3 (effective batch 6), AMP off.

1. **AMP is a net loss on this card.** No tensor cores, so fp16 convolution buys $\sim$5 % memory for a 2.8–3.2$\times$ step-time penalty — the opposite of the usual advice, and only measurement shows it.
2. **The binding constraint is the Windows TDR watchdog, not VRAM.** This is the display GPU; a kernel over $\sim$2 s is killed and the CUDA context destroyed. Batch 4 crosses it in both runs.
3. **Step timing is not reproducible to better than** $\sim$50 % on a machine whose GPU also drives the desktop. Batch 3 measured 1432 ms and 2193 ms minutes apart with identical peak memory, so it was rejected despite fitting.
4. **Encoder size barely matters**: `pvt_v2_b0` and `pvt_v2_b2` reach similar throughput, because the loss — not the encoder — dominates the step (§7).

The batch-6 row reporting 4268 MiB on a 4096 MiB card is not an error: WDDM allows oversubscription into host RAM, so it "fits" while thrashing.

5 Stage C — the supervision ablation

Four arms, one network, one dataset, one schedule; only `--supervision` differs. 20 epochs, effective batch 6, seed 2222, validation every 2 epochs.

Arm	Isolates	Best Dice	Peak VRAM	Wall	Status
`last_layer`	no deep supervision	0.6764	892.7	364	completed
`deep_supervision`	uniform per-scale	**0.7979**	924.0	368	completed
`mutation`	mixing, unweighted	0.7788	1067.0	405	completed
`lomix`	learnable weights	0.7225†	1768.8	594	**diverged (NaN) ep. 13**

VRAM in MiB, wall clock in minutes. †best value before divergence.

Validation curves (Dice at each validation):

Arm	Curve
`last_layer`	0.676, 0.588, 0.562, 0.593, 0.569, 0.605, 0.566, 0.578, 0.618, 0.598
`deep_supervision`	0.659, 0.613, 0.607, 0.684, 0.626, 0.727, 0.759, 0.751, 0.741, **0.798**
`mutation`	0.563, 0.643, 0.698, 0.701, 0.683, 0.768, 0.772, 0.757, 0.773, **0.779**
`lomix`	0.609, 0.676, 0.708, 0.703, **0.7225**, 0.711, 0.000, 0.000

Two things are visible immediately. `last_layer` peaks at its *first* validation and then decays — consistent with the training-mode defect in §6.3. And at this budget the paper's ordering does **not** reproduce: uniform deep supervision beats unweighted mixing, which beats the learnable-weight arm, which then diverges. Under-training is the most likely explanation and is not separable from the result at 20 epochs; §8 lists what would settle it.

5.1 The `lomix` arm diverged

At iteration 14900, epoch 13, `loss`, `deep_supervision_loss` and `mutation_loss` all became NaN simultaneously. Every subsequent validation returned Dice 0.000. The best pre-divergence value was 0.7225 at epoch 9.

This was not a hardware event: fp32 throughout, no OOM, no watchdog reset, and the weights in `best.pth` are all finite. The cause is in the loss (§6.2). An earlier `lomix` attempt survived to epoch 19 only because a machine shutdown had forced a restart at epoch 11 with a fresh optimizer, which reset the state that was accumulating toward overflow — that run is reported separately as **not comparable**, because a resume without optimizer state is a different experiment.

5.2 Learned mixing weights

All weights initialise at softplus(0) = 0.6931. Final values from the `lomix` arm before divergence:

Group	Value	Movement from initialisation
Per-scale p_4, p_3, p_2, p_1	0.379, 0.386, 0.392, 0.400	all fell to ≈0.39, nearly uniform
`mul` (11 subsets)	0.434–0.464	**up-weighted**
`add` (11 subsets)	0.417–0.459	**up-weighted**
`wf` (11 subsets)	0.385–0.397	mid
`concat` (11 subsets)	0.3755–0.3770	**collapsed, identical across all 11**

The per-scale weights barely differentiate at 20 epochs, which suggests LoMix's weighting needs far more budget to specialise than a reduced run provides. The `concat` collapse turned out to have a concrete cause — §6.2.

6 Defects found in the released codebase

All three were found by running the code, and all three are reproducible.

6.1 The training script cannot start

`train_synapse_lomix.py:10` imports `PVT_CASCADE`, which is defined nowhere in the repository — `lib/networks.py` defines only `EMCADNet`, and the only other reference is a commented-out model line. As shipped, the script fails at import before parsing an argument. EMCAD's equivalent script imports only `EMCADNet`, so this looks like leftover copy-paste.

6.2 Two defects in the combinatorial loss

Both verified empirically by `tools/diagnose_loss_ops.py` on the trained checkpoint, not merely read off the source.

`concat` rebuilds a random convolution on every forward pass. In `trainer.py`:

```
elif op == 'concat':
    cat = torch.cat([output_maps[c] for c in comb], dim=1)
    conv = nn.Conv2d(cat.size(1), self.num_classes, kernel_size=1).to(device)
    mutated = conv(cat)
```

The conv is created inside `forward`, randomly initialised each call, never registered as a submodule and never given to the optimizer. Evidence: calling the loss twice with identical inputs under `no_grad` gives

Operation	Max $\lvert a-b\rvert$	
`add`	0.0	deterministic
`mul`	0.0	deterministic
`wf`	0.0	deterministic
`concat`	**5.41**	**non-deterministic**

and none of the loss module's 66 parameter tensors belongs to it (the conv parameters that *are* registered all belong to `weighted_fusion_modules`, i.e. the `wf` operation). So the `concat` branch cannot learn and contributes a fresh random projection each step. This explains §5.2 exactly: `concat`'s learnable weights have no gradient signal to differentiate on, so they collapse to a common value.

`mul` multiplies raw logits, so a k-map combination is a k-fold product growing like $|\text{logit}|^k$. With the trained checkpoint's logits ($\max|p| = 6.16$):

Operation	$k=2$	$k=3$	$k=4$
`add`	12.0	17.9	23.6
`mul`	**36.2**	**211.0**	**1214.0**
`wf`	6.1	6.1	5.9
`concat`	2.8	2.9	1.9

Cross-entropy and Dice saturate on values of that size and the gradient overflows. Nothing clips or normalises the product. This is the mechanism behind the epoch-13 NaN, and it is a property of the method as released, not of this hardware — the magnitudes above are computed on the model's own trained logits.

6.3 Training continues in eval mode after the first validation

`trainer.py` calls `model.train()` once *before* the epoch loop; `inference()` calls `model.eval()` and runs at the end of every epoch; nothing switches back. From epoch 1 onward the decoder's `BatchNorm2d` layers normalise with frozen running statistics and stop updating them, and dropout/drop-path are inactive. The same pattern is present in the EMCAD and G-CASCADE trainers.

This was **left intact** — the published results were produced by this code, so preserving the behaviour is what replication means. The patch adds `--restore_train_mode` (default off) so the alternative can be run as a separately labelled experiment. Not run here for lack of GPU time.

6.4 Windows portability

`worker_init_fn` is defined as a local function inside `trainer_synapse`. Windows starts DataLoader workers with `spawn`, which pickles the callable, and local functions are not picklable: `AttributeError: Can't pickle local object 'trainer_synapse.<locals>.worker_init_fn'`. Linux uses `fork` and is unaffected. Fixed by hoisting the function to module scope and binding the seed with `functools.partial`; seeding behaviour is unchanged.

7 Training cost of the supervision — measured

Same network, same batch, same data; only `--supervision` differs (`pvt_v2_b2`, 224^2, batch 3, AMP off, `logs/memfit_Ishad_*.json`):

Arm	Peak train VRAM	Step time	vs `last_layer`
`last_layer`	1008.2 MiB	649 ms	1.00×
`deep_supervision`	1053.7 MiB	708 ms	1.09×
`mutation`	1268.3 MiB	938 ms	1.45×
`lomix`	2289.6 MiB	1649 ms	**2.54**×

LoMix costs 2.54× step time and 2.27× training memory over a single-logit baseline while adding nothing at inference, and roughly two thirds of that gap comes from the learnable-weight arm alone (`mutation` → `lomix`, 938 → 1649 ms). The paper reports the inference cost; this training cost is not reported, and on constrained hardware it decides whether the method is runnable at all.

8 Discrepancies and what would settle them

1. **The ordering does not reproduce at 20 epochs.** `deep_supervision` (0.798) > `mutation` (0.779) > `lomix` (0.723, diverged). Most likely under-training: §5.2 shows the mixing weights had barely differentiated from their common initialisation, so the mechanism that is supposed to create LoMix's advantage had not yet engaged. Settled by a full-schedule run, which this hardware cannot deliver in the available time.
2. **The `lomix` arm has no completed run.** It needs either gradient clipping or a bounded `mul` (e.g. multiplying softmax probabilities rather than raw logits) to finish. Either is a change to the method and would have to be applied to every arm to keep the comparison fair.
3. **Single seed, one dataset, one encoder, no statistical testing.** The `mutation`/`deep_supervision` gap (0.779 vs 0.798) is small enough that one seed cannot separate them.

8.1 Side-by-side with the published numbers

The paper's Table 1 reports exactly these four arms on exactly this backbone (PVT-EMCAD-B2, Synapse 8-organ), which makes the comparison direct. The left block is **the paper's**, averaged over at least three runs on an RTX A6000; the right block is **mine**, one seed at 20 epochs on a GTX 960. These are not like-for-like and the gap should not be read as a contradiction.

	Paper (Table 1)			This work
Arm	**DICE**	**HD95**	**mIoU**	**DICE**
`last_layer` (LL)	80.9	22.9	71.2	67.6
`deep_supervision` (DS)	82.9	19.7	73.8	**79.8**
`mutation`	83.6	15.7	74.7	77.9
`lomix`	**85.1**	**14.9**	**76.4**	72.3†

All values are percentages. †best before divergence at epoch 13, not a completed run.

Three things follow.

1. **The published ordering is monotone (LL < DS < mutation < LoMix); mine is not.** The paper reports +4.2 DICE for LoMix over LL, +2.2 over DS and +1.5 over unweighted mutation, with two-sided Wilcoxon signed-rank tests confirming significance over LL and DS. My runs reproduce the sign of the LL → DS step (+12.2, larger than the paper's +2.0) but not the DS → mutation → LoMix steps.
2. **Absolute values are uniformly lower**, by 3–13 DICE. This is what 20 epochs against a converged schedule looks like, and my best arm (79.8) sits just below the paper's *weakest* arm (80.9) — consistent with under-training rather than with a broken setup.
3. **The margins I would need to resolve are small.** The paper's DS → LoMix gap is 2.2 DICE, established over at least three runs with a signed-rank test. A single seed at 20 epochs cannot separate effects of that size, which is why §8.1 treats my ordering as unresolved rather than contradictory.

9 Reproducing this

```
git clone https://github.com/irtiza1999/LoMix-Reproducibility-Study.git
cd LoMix-Reproducibility-Study
powershell -ExecutionPolicy Bypass -File setup\01_create_env.ps1
conda activate lomix
cd repos\LoMix
python ..\..\tools\bench_efficiency.py
python ..\..\tools\memfit.py
cd ..\..
python tools\check_data.py --repo repos\LoMix        # must print DATA OK
python tools\apply_low_vram_patch.py --repo repos\LoMix
.\run_ablation.ps1                                   # ~18 h, 4 arms
python tools\collect_results.py --stamp <stamp>
python tools\diagnose_loss_ops.py --ckpt <best.pth>  # the section 6.2 evidence
```

All of it is in the harness repository: raw run logs and per-sweep manifests in `logs/`, the patch script and the untouched `trainer.py.orig` for a line-level diff in `repos/LoMix/`, and the analysis tools (`collect_results.py`, `diagnose_loss_ops.py`, `memfit.py`, `bench_efficiency.py`, `check_data.py`, `smoke_test.py`) in `tools/`. Every table in this document can be regenerated from those logs with `collect_results.py`.

Runs are checkpointed after every epoch with model, optimizer, scaler, `best_dice`, `iter_num` and epoch, written atomically, so `-Resume` continues a shutdown-interrupted run identically rather than restarting the optimizer.